

第十三节 · Spec-Kit 教程：原理 · 用处 · 用法 · 时机

把 Spec-Kit 一次讲清楚。

一份实用的工具教程——原理、有什么用、怎么用、什么时候用。

这节，讲清楚四件事

Spec-Kit 不是 AI agent，是套在 AI agent 之上的「工作流层」。

01

原理

它怎么工作

02

有什么用

解决什么问题

03

怎么用

命令与流程

04

什么时候用

适合什么场景

一个 CLI + 一套 slash command，支持 20+ AI agent。

装一次，用 Claude Code、Copilot、Cursor 都行。它不替代 AI agent，而是给它套上一套结构化的工作流——本节就把这套工作流讲透。

为什么需要 SDD

「给 AI 模糊需求、直接生成代码」——vibe coding 在大项目上会出事。

需求漂移

跨会话进行，AI 每次重新理解，决策被遗忘

架构失控

AI 只求「看起来能跑」，迭代几次变一团乱麻

回退困难

没有正式需求文档，错了不知该回到哪

无法协作

每个人提示词理解不同，产出风格不一致

仅靠 **prompt** 不够了。

要在 **prompt** 之上加一层结构化的「需求层」——这就是 SDD，而 Spec-Kit 是它最纯粹的实现。

原理 ①: spec-as-source

spec 是唯一源头，其它一切都从它派生。

第一步：初始化

在当前仓库里执行（`--here` 用当前目录，`--force` 因为仓库非空跳过确认）：

```
specify init --here --integration claude --force
```

spec.md

唯一源头

plan.md

技术方案

tasks.md

任务列表

code

代码

spec 变了，下游 plan / tasks / code 全部重新生成

spec 是项目的唯一事实来源，不是 chat 历史。

它是仓库里的正式文件，跟代码一起版本管理——这是 Spec-Kit 和「随便聊聊让 AI 写」的根本区别。

Thinking

脚手架就位了。有个关键点：`.specify/memory/constitution.md` 现在还是空模板（全是 `[PROJECT_NAME]` 占位符）——所以第一件事就是立宪。

命令执行顺序与作用

按这个顺序走，每个都是斜杠命令（也可直接对我说“执行 xxx”）：

顺序	命令	作用	必需？
1	<code>/speckit-constitution</code>	立项目宪法：把技术栈、铁律写进 <code>constitution.md</code> ，后续所有阶段都受它约束	✓ 先做
2	<code>/speckit-specify</code>	写规范：用自然语言描述某个特性「做什么、为什么」，自动建分支 + <code>specs/xxx/spec.md</code> （不谈实现）	✓ 每个特性
3	<code>/speckit-clarify</code>	消歧：对 spec 里模糊的地方提最多 5 个精准问题，把答案写回 spec，降低返工	★ 强烈建议
4	<code>/speckit-plan</code>	定方案：技术方案、架构、选型 → <code>plan.md</code> + 设计文档	✓
5	<code>/speckit-checklist</code>	质量清单：按需求生成一份验收/质量检查清单	可选
6	<code>/speckit-tasks</code>	拆任务：把 plan 拆成依赖有序的 <code>tasks.md</code>	✓
7	<code>/speckit-analyze</code>	一致性检查：交叉核对 spec / plan / tasks 有没有矛盾、遗漏（非破坏性）	★ 建议
8	<code>/speckit-implement</code>	执行：按 <code>tasks.md</code> 逐个写代码	✓

辅助命令：

- `/speckit-converge` — 拿现有代码库对照 `spec/plan/tasks`，把「还没做的」补成新任务，适合中途接手或迭代。
- `/speckit-taskstoissues` — 把 `tasks` 转成 GitHub issues。

原理 ②：三大设计支柱

这三点，让 AI agent 从「凭感觉」变成「按规格干活」。

支柱一

constitution 项目宪法

把非协商原则（技术栈、规范、边界）固化成文件，所有命令都引用，AI 每次都看到。

支柱二

4 阶段闭环

specify → plan → tasks → implement，每阶段有正式 artifact，前一阶段输出是后一阶段输入，不跳步。

支柱三

analyze 防漂移

跨 artifact 一致性检查：constitution / spec / plan / tasks 是否对齐，发现漂移立刻修。

constitution 钉死方向、4 阶段保证不跳步、analyze 防止 spec 和代码渐渐对不上。

原理 ③：产物长什么样

specify init 生成 .specify/ 工作区——四个正式文件，跟代码一起版本管理。

constitution.md

项目宪法：非协商原则（写一次定下来，全程不轻易改）

spec.md

需求规格：user story + acceptance criteria + 功能边界

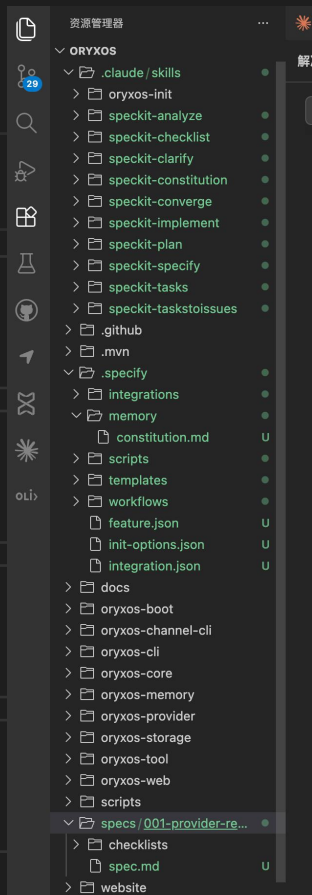
plan.md

技术方案：技术栈选型、模块划分、关键技术决策

tasks.md

任务清单：按 user story 分组、标好依赖与可并行

四个文件是「人和 AI 之间的契约」，也是项目决策的留底——可审查、可共享、可回溯。



有什么用：从作坊到工程

把「想清楚」变成正式产物——这是 AI 编程从作坊到工程的关键一步。

对治乱来

管住 **vibe coding**

不再凭感觉聊天，需求
漂移、架构失控被 **spec**
兜住。

立契约

spec 是人与 AI 的
契约

双方都遵守的依据，AI
不再靠概率猜你的意图。

可追溯

决策持久化、可审
计

所有决策有 **artifact** 留底，
跟代码一起版本管理。

可演进

长期项目不丢上下
文

跨会话切换不丢决策，
spec 跟代码持续同步演
进。

一句话：没有 SDD 的 AI 编程，跟没有版本控制的传统编程是同一个问题——能跑，但不可持续。

怎么用 ①：安装与命令体系

specify init 起手；命令以 /speckit. 前缀触发。

6 个核心命令

必走的闭环

- constitution: 写项目宪法
- specify: 需求 → spec (user story)
- plan: spec + 宪法 → 技术方案
- tasks: plan → 有序任务列表
- implement: 按 tasks 生成代码

3 个可选命令

加强代码质量

- clarify: specify 后反问澄清模糊点
- analyze: implement 前查一致性 (防漂移)
- checklist: 自定义质量校验清单
- (taskstoissues: 任务转 GitHub issues)

命令语法不用背——理解每个命令解决什么问题，跑一遍就熟。

怎么用 ②：典型 workflow

从初始化到 implement，一条不跳步的流水线。



没有跨阶段 shortcut，没有「凭感觉跳过 plan 直接写代码」。每个 task 写完人工 review 再 commit。

怎么用 ③：一条命令长什么样

拿两个最关键的命令，看真实的输入和输出。

/speckit.specify

输入：一句话 / 一段粗略需求

输出：spec.md（约 800 行）——背景、user story、acceptance criteria、非目标（明确说不做什么）

/speckit.tasks

输入：spec.md + plan.md

输出：tasks.md——几十个有序任务，按 user story 分组，标好依赖关系和可并行项

你只写第一份 spec、检查 plan、最后 review 代码；中间的拆任务和执行，AI 自动跑。

什么时候用：它是「重武器」

强在强制完整流程、产出结构化 artifact；代价是流程开销大。

适合

威力巨大

- greenfield 中到大型项目
- 需求清晰，上游有文档或决策
- 用 AI agent 做主体开发
- 团队要学 SDD 工程方法论

不适合

开销大于收益

- 小 feature、单文件、快速原型
- 大型 brownfield 改造（上下文超限）
- 探索性研究（需求未定会返工）
- 个人小实验（6+3 命令太重）

用在合适的场景威力巨大，用在不合适的场景就是负担——工具匹配工作性质。

课前，先动手跑一遍

不用记命令——装一遍、跑一遍，概念立刻就立住了。

STEP 1

装 Spec-Kit

`specify init` 在一个空项目里初始化，看 `.specify/` 长什么样。

STEP 2

写 constitution

按格式写一份非协商原则，体会「钉死方向」的感觉。

STEP 3

跑 `specify` → `plan` → `tasks`

把一个小需求喂进去，看它怎么拆成任务清单。

► 下一节《讲解 Spec-Kit 拆解后的任务》（演示课）——现场看拆出来的内容怎么用。